CHAPTER 15

Choosing a Model

In this chapter

Introduction: A $60,000 Decision Made on 100 Examples . . . 3
15.1 The Question Is Not "Which Model Is Best" . . . 4
15.2 What the Official Guidance Says . . . 5
15.3 A Candidate Is a (Model, Effort) Pair . . . 6
15.4 The Evaluation Set Is the Whole Job . . . 6
15.5 Accuracy Is Not the Metric . . . 7
15.6 The Number Nobody Writes Down . . . 7
15.7 Two Frontiers . . . 8
15.8 The Floor Rule . . . 9
15.9 Sizing the Evaluation . . . 10
15.10 Speed . . . 10
15.11 Constraints That Override Measurement . . . 11
15.12 Visual Analogies . . . 12
15.13 Architecture: Selection Is a Pipeline, Not a Meeting . . . 12
15.14 When to Re-open the Decision . . . 13
15.15 Code Walkthrough: Selection as Arithmetic . . . 14
15.16 Mini Project: The Selection Harness . . . 15
15.17 The Simulation, and Why It Is One . . . 16
15.18 Panel A — Ranked by Accuracy . . . 17
15.19 Panel C — The Two Frontiers . . . 17
15.20 Panel B — Cost per Correct Answer . . . 18
15.21 Panel D — One Evaluation of One Hundred . . . 18
15.22 Panel E — Two Thousand Replications . . . 19
15.23 Panel G — What Does a Wrong Answer Cost You? . . . 20
15.24 Panel F — The Effort Axis Versus the Model Axis . . . 21
15.25 Evaluation and Deployment . . . 22
15.26 Hands-on Labs . . . 22
15.27 Lab Solutions . . . 23
15.28 Best Practices . . . 23
15.29 Common Mistakes . . . 24
15.30 Performance Considerations . . . 25

15.31 Security Considerations 26
15.32 Real Production Examples 26
15.33 Certification Notes 27
15.34 Twenty Practice Questions 28
15.35 Answers and Explanations 28
15.36 Ten Practical Exercises 30
15.37 Five Challenge Labs 31
15.38 Ten Interview Questions 31
15.39 Summary 31
15.40 Glossary 33
15.41 Further Reading 34

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. State the four criteria the official guidance puts on a model decision, and the two documented strategies for choosing a starting point.
2. Explain why a **candidate is a (model, effort) pair**, and why comparing models at fixed effort searches a line through a plane.
3. Build an evaluation set that can decide the question you are asking, and **size it** for the difference that would change your mind.
4. Rank candidates by **cost per correct answer**, and say precisely when that metric is wrong.
5. Compute the **cost of a wrong answer** and the failure price at which a dearer model becomes the cheaper one.
6. Distinguish the **economic frontier** from the **defensible frontier**, and delete dominated options without argument.
7. Apply the **floor rule** — the cheapest candidate whose lower confidence bound clears your quality bar — and defend the choice in a review.
8. Recognise the constraints that override measurement entirely, and know when to re-open a decision you have already made.

NOTE

Prerequisites

Chapter 8 is load-bearing throughout: Wilson intervals, evaluation-set sizing, and the rule that a difference whose intervals overlap has not been established. **Chapter 11** for the latency model and the rate card. **Chapter 12** for benchmark scepticism and the sample-size arithmetic. **Chapter 13** for `effort`, thinking economics, and the rule that settings do not transfer between models. **Chapter 14** for the catalogue this chapter selects from, and for the registry that makes the answer one line to apply.

Chapter 14 built the catalogue. This chapter is the decision procedure, and it is deliberately a *procedure* rather than a recommendation, because a recommendation ages and a procedure does not.

Introduction: A $60,000 Decision Made on 100 Examples

A team is choosing a model for a document-classification service. They have done the responsible thing and built an evaluation set: one hundred examples, hand-labelled by two people, disagreements resolved by a third. It took a week. Nobody in the industry would call it inadequate.

They run four candidates. The scores come back 91%, 90%, 90%, 85%. The 91% belongs to the most expensive model in the family. Somebody puts the four numbers on a slide, the slide is convincing because four numbers in descending order are always convincing, and the decision takes eleven minutes.

Here is what the same hundred examples say when you print the intervals:

```
          candidate     acc             95% CI   width
        opus-5/high    0.91 [0.8377,0.9519]  0.1142
     fable-5/medium    0.90 [0.8256,0.9448]  0.1191
```

```
        fable-5/high    0.90 [0.8256,0.9448]  0.1191
     sonnet-5/medium    0.85 [0.7672,0.9069]  0.1398
```

Every interval is about eleven points wide, and every interval overlaps every other one. The winner is **established better than three of the nine candidates it was compared against**, and none of those three is the one it beat by a point. The eleven-minute decision committed roughly $60,000 a month to a difference the evaluation could not resolve.

That is not the interesting failure, though. The interesting failure is the one underneath it, and it is a failure of framing rather than of statistics. The team compared four *models*. There were ten *candidates*, because each model can be run at several effort levels, and the effort axis crosses the model axis: in the measurements later in this chapter, **Sonnet 5 at high effort beats Opus 5 at low effort on accuracy and costs 42% less**. A comparison that holds effort fixed has already thrown away the part of the search space where the interesting answers live. Anthropic's own guidance says so directly: *tuning effort is often a better lever than switching models.*

And underneath *that* is the number nobody wrote down. Nobody in the room asked what a misclassification costs. It turns out to be the number that decides everything: in section 15.23's measurements, the cheapest total-cost option is Haiku 4.5 when a wrong answer is free, Sonnet 5 at medium effort when it costs five pence, Opus 5 at high effort when it costs a pound, and Fable 5 at high effort when it costs five pounds. Same accuracy figures, same rate card, four different winners.

This chapter is a procedure that gets those three things right: the candidates, the intervals and the failure cost. It ends with a rule you can defend in a meeting, and a measurement of how much money the rule saves against the eleven-minute version — which, on the workload simulated here, is **$23,179 a month**, and that is the value of a bigger evaluation set rather than of a better model.

15.1 The Question Is Not "Which Model Is Best"

"Best" is not a property of a model. It is a property of a model, a task, a metric, a budget, a latency target and a tolerance for being wrong — and changing any of those six can change the answer.

The question that has an answer is:

KEY TAKEAWAYS

The selection question

What is the cheapest candidate whose quality I can defend against my bar, on my task, at my volume, within my constraints?

Every word is doing work. *Cheapest* because among candidates that clear the bar, price is the only remaining discriminator. *Defend* because a point estimate is not a finding. *My task* because Chapter 12 established that a leaderboard measures somebody else's distribution. *My volume* because the ranking by unit price and the ranking by monthly bill are different rankings once failure costs enter. *Within my constraints* because some requirements are not tradeable, and section 15.11 lists them.

Notice what the question does not ask. It does not ask which model scored highest, because that question is answered by the most expensive model roughly by construction and tells you nothing about whether the extra money bought anything. It does not ask which model is most capable in general, because

Chapter 12 spent a chapter explaining why that question is badly posed.

15.2 What the Official Guidance Says

Before the procedure, the documentation, quoted rather than paraphrased. The platform's *Choosing the right model* page frames the decision as balancing **capabilities, speed and cost**, and adds a fourth criterion that is easy to miss:

> **Effort:** Recent Opus and Sonnet models support an effort parameter that trades intelligence for latency and cost within a single model. **Tuning effort is often a better lever than switching models.**

That sentence is the whole of section 15.3 and a large part of this chapter.

The documentation then gives **two starting strategies**, and it is worth knowing both because they suit different situations and an exam may ask which fits a described scenario.

Option 1: start efficiency-first. Begin with Claude Haiku 4.5, test thoroughly, evaluate whether performance meets requirements, and upgrade only for specific capability gaps. Documented as best for initial prototyping, tight latency requirements, cost-sensitive implementations, and high-volume straightforward tasks.

Option 2: start capability-first. Implement with Claude Opus 5, optimise prompts for that model, evaluate, then increase efficiency by lowering effort or downgrading models as the workflow matures. Documented as best for complex reasoning, scientific and mathematical applications, tasks requiring nuanced understanding, applications where accuracy outweighs cost, and advanced coding and high-autonomy agentic work.

The published selection matrix:

When you need	Consider starting with	Example use cases
The highest available capability	Claude Fable 5	Long-running agents, deep reasoning, long-horizon agentic tasks, advanced research
Complex agentic coding and enterprise work	Claude Opus 5	Multihour autonomous coding agents, large-scale refactoring, complex systems engineering, vision-heavy workflows, computer use
Frontier intelligence at scale for coding, agents and enterprise workflows	Claude Sonnet 5	Code generation, data analysis, content creation, visual understanding, agentic tool use
Near-frontier performance with lightning-fast speed at the most economical price	Claude Haiku 4.5	Real-time applications, high-volume intelligent processing, cost-sensitive deployments needing strong reasoning, sub-agent tasks

And on how to decide whether to change models, the documentation is unambiguous about what matters most:

> Create benchmark tests specific to your use case — **having a good evaluation set is the most important step in the process.**

One further fact belongs under *speed*, because it is a real option and it is easy to overlook: **Claude Opus 5 and Claude Opus 4.8 support fast mode** (research preview), delivering **up to 2.5× higher output speed at premium pricing**. Chapter 11 priced that premium at $10/$50 per MTok, matching Fable 5's rate, and noted that fast mode is not available with the Batch API. It is the one lever that buys latency without reducing the token count.

WARNING

Guidance is a starting point, not a decision

Every row of that matrix says "consider starting with". The documentation's own final section is about building benchmark tests and comparing on your data, and the rest of this chapter is how to do that without fooling yourself. Treat the matrix as a way to pick the first three candidates rather than as an answer.

15.3 A Candidate Is a (Model, Effort) Pair

This is the reframing that makes the rest work.

DEFINITION

Candidate

A **candidate** is a specific configuration you could ship: a model **and** an effort level, and in principle a thinking setting and a test-time compute strategy too. It is the unit a selection procedure compares, because it is the unit you deploy.

Four current models with the effort levels they support give ten to fifteen plausible candidates before you have varied anything else. Comparing four models at their default `high` effort explores four of them, and the default is the most expensive setting on every model that has one — which is Chapter 13's observation that silence is a decision to spend.

The measurements in section 15.24 show why this matters. Two rows from that table:

Cheaper	Dearer	Accuracy gain	Extra $/month	Established?
`sonnet-5/high`	`opus-5/low`	**−0.0059**	+$10,800	no
`opus-5/high`	`opus-5/xhigh`	**−0.0014**	+$14,000	no

In both rows the dearer option is *less* accurate. Sonnet 5 at high effort beats Opus 5 at low effort on both axes at once, which means `opus-5/low` is dominated and can be deleted from the conversation. And `opus-5/xhigh` is dominated by `opus-5/high` from the same model — more effort, more money, no measurable gain, which is Chapter 13's diminishing returns showing up as a line item.

Neither of those findings is available to a comparison that holds effort fixed. They are not corner cases; they are two of the ten candidates.

15.4 The Evaluation Set Is the Whole Job

Chapter 8 built evaluation sets and Chapter 12 explained why somebody else's benchmark cannot decide your question. This section is the short version of both, because everything downstream multiplies by it.

Your task, your distribution, your metric. Sampled from production traffic if you have it, including the hard slices rather than only the median case. If your traffic is 5% adversarial and your evaluation set is 0% adversarial, you are choosing a model for a workload you do not have.

The metric is chosen before you see any scores. Chapter 12's fifth benchmark trap is choosing the metric after the results arrive, and it is the one people commit without noticing.

It is private and it is held out. A public benchmark may be in the training data, which Chapter 12 called contamination and which turns memorisation into apparent capability.

It is sized for the difference that would change your mind. This is the step teams skip, and it is arithmetic rather than judgement. Section 15.9 does it.

It is version-controlled and re-runnable. You will re-run it when a model is deprecated, when a new one launches, and when Chapter 14's serving infrastructure moves under a pinned ID. A one-off spreadsheet is not an evaluation set; it is a memory of one.

15.5 Accuracy Is Not the Metric

Ranking candidates by accuracy produces a table that answers a question nobody asked. Two better metrics, in increasing order of honesty.

Cost per correct answer. Divide the cost of a request by the probability it is right. It reorders the table almost completely — in section 15.20 the accuracy ranking and the cost-per-correct ranking are close to exact inversions of one another.

It is also frequently misread, so here is the correction stated plainly. Cost per correct answer does **not** simply reward accuracy. It rewards the better *ratio*, and price ratios in this family are larger than accuracy ratios. Haiku 4.5 at 50% accuracy still beats Opus 5 at 95% on this metric, because the price gap is fivefold and the accuracy gap is twofold. There is a test asserting exactly that, because it is counter-intuitive and important.

The deeper problem is the assumption hiding inside the division:

> WARNING
>
> **Cost per correct answer assumes a wrong answer is free**
>
> Dividing by accuracy is only the right arithmetic when you can discard a wrong answer at no cost — when you have a cheap verifier, in Chapter 13's vocabulary, and can simply try again. If a wrong answer costs a retry, a human review, a chargeback or a customer, then failures are a line item and they belong in the sum rather than in the denominator.

15.6 The Number Nobody Writes Down

So write it down. The honest total is:

```
total cost per request  =  inference cost  +  (1 − accuracy) × failure cost
```

where **failure cost** is what one wrong answer costs downstream. A retry at the same price. A human reviewer at forty pounds an hour spending four minutes, which is £2.67. A support contact. A chargeback. A churned account. Nobody knows this number exactly and everybody can bound it, and a bounded estimate beats the implicit assumption of zero that a raw accuracy table encodes.

Rearranging gives the quantity that actually decides the question:

DEFINITION

Break-even failure cost

For a cheaper candidate `c` and a dearer candidate `d`:

```
F* = (cost_d − cost_c) / (accuracy_d − accuracy_c)
```

Below `F*` per failure, the cheaper candidate is cheaper overall. Above it, the dearer one is. It is the price of an accuracy point, read as a threshold rather than a rate, and it converts an argument about model quality into a question about your own operations that your own team can answer.

Section 15.23 computes these against Haiku 4.5 on the simulated workload, and the numbers are small enough to be sobering:

Upgrade from Haiku 4.5 to	Break-even failure cost
`sonnet-5/medium`	**$0.049**
`sonnet-5/high`	**$0.066**
`opus-5/high`	**$0.183**
`fable-5/high`	**$0.377**

If a wrong answer costs you more than five pence to deal with, Sonnet 5 at medium effort is cheaper than Haiku 4.5 — not more accurate for more money, *cheaper*. If it costs more than about eighteen pence, so is Opus 5 at high effort. Almost any workload with a human anywhere in the loop clears every one of those thresholds, which is a strong argument against the reflexive cheapest-model choice, arrived at without appealing to quality at all.

15.7 Two Frontiers

With candidates measured and priced, most of them can be eliminated mechanically.

DEFINITION

The economic frontier

Candidate `a` **dominates** `b` when `a` is at least as accurate and costs no more, and is strictly better on one of the two. The **economic frontier** is the set of candidates nothing dominates. Everything else can be deleted without argument, because there is a strictly better option at no extra cost.

On the ten simulated candidates, two are dominated: `opus-5/low` (beaten by `sonnet-5/high`) and `opus-5/xhigh` (beaten by `opus-5/high`). Eight survive, and no amount of preference can rescue the two that do not.

But the economic frontier is built from point estimates, and a point estimate is not a finding. The stronger question inverts it:

DEFINITION

The defensible frontier

A candidate stays on the **defensible frontier** only if it is **established better than everything cheaper than it**. A candidate whose extra cost buys an improvement you cannot demonstrate is a candidate you cannot justify in a review.

The cheapest candidate is always on this frontier — it has nothing cheaper to beat. On a small evaluation it is frequently the *only* thing on it, and that is a finding about your evaluation rather than about the models.

Seven of the ten survive on twenty thousand examples. That last clause is the one to hold on to: the defensible frontier is a function of how much you measured, and it shrinks toward a single point as your evaluation set shrinks.

15.8 The Floor Rule

The decision itself is two lines.

KEY TAKEAWAYS

The floor rule

Choose the cheapest candidate whose lower confidence bound clears your quality floor.

Use the *lower bound*, not the point estimate. The point estimate asks "did this quality happen to come up in my sample?". The lower bound asks "can I defend this quality?", which is the question a review, a regulator and a post-incident analysis will all ask.

Two consequences, and the second is the useful one.

The rule can return nothing. If no candidate's lower bound clears the floor, the honest output is "not established" — and the right next action is a larger evaluation set, not a more expensive model. In the simulation at fifty examples the rule declines to choose 6.3% of the time.

A small evaluation forces you to overbuy. This is the non-obvious result and it falls straight out of the arithmetic. On a hundred examples the intervals are eleven to seventeen points wide, so only the top-scoring candidates have a lower bound above an 0.80 floor — and the rule dutifully buys one of

them. As the evaluation grows the intervals narrow, cheaper candidates become defensible, and the bill falls. Section 15.22 measures it over two thousand replications:

Evaluation size	Mean monthly cost under the floor rule
50	$35,172
100	$23,172
400	$13,443
2,000	$11,993

$23,179 a month separates the fifty-example evaluation from the two-thousand-example one, on identical models with identical true accuracies. Nothing about the models changed. What changed is what you could prove, and what you could prove is what you were allowed to stop paying for. That is the argument for evaluation, denominated in money rather than in virtue.

15.9 Sizing the Evaluation

The arithmetic from Chapter 12, restated as a design step. Examples per arm needed to resolve a gap of δ around accuracy p:

```
n  ≈  2 z² p (1 − p) / δ²          z = 1.96 for 95%
```

Gap you want to resolve	Around 88%	Around 90%
4 points	508	433
2 points	2,029	1,729
1 point	8,114	6,915

Halving the gap quadruples the sample. That is the single most useful fact in evaluation design, and it cuts both ways: resolving a one-point difference is usually not worth eight thousand labelled examples, and the right response is to decide that a one-point difference does not change your decision and stop trying to resolve it.

Turn it around and it becomes a design question with a cheap answer. *What difference would change my mind?* If four points would change it and one point would not, you need about five hundred examples, not eight thousand — and five hundred is a week of work rather than a quarter.

There is a limit to how far this goes, and section 15.24 shows it: even on **twenty thousand** examples, a 0.4-point gap between `opus-5/high` and `fable-5/medium` is *not established*. The correct reading is not that you need more data. It is that a difference you cannot establish on twenty thousand examples is a difference you should stop paying $22,000 a month for.

15.10 Speed

Cost has a rate card and speed does not, so speed is where selection procedures get sloppy. Three points, from Chapter 11.

Latency is `prefill + output_tokens × per_token_decode`, and decode is 82–99% of wall clock. The documented latency ordering — Fable 5 slower, Opus 5 moderate, Sonnet 5 fast, Haiku 4.5 fastest — is a per-token statement. Your users experience a per-request one, and effort moves the token count by up to 5×. **A faster model at high effort can be slower end-to-end than a slower model at low effort.** Measure the request, not the tier.

Streaming changes perception, not physics. Time-to-first-token is a product decision; total generation time is unchanged. Chapter 13 added that `display: "omitted"` shortens time-to-first-*text*-token by skipping the streaming of thinking tokens, at no cost saving.

Fast mode is the one lever that buys latency without cutting tokens. Up to 2.5× higher output speed on Opus 5 and Opus 4.8, at premium pricing ($10/$50), not available with the Batch API. It belongs in the candidate list for latency-bound workloads, and it is the correct answer when the alternative is dropping to a weaker model purely for speed.

And the asymmetric case worth naming: if latency does not matter, the **Batch API halves every price**, which moves the whole frontier and frequently makes the top model affordable. "We cannot afford Fable 5" and "we cannot wait" are different sentences, and only one of them is usually true.

15.11 Constraints That Override Measurement

Some requirements are not on the frontier because they are not tradeable. Check these before you measure anything, because they eliminate candidates for free.

Constraint	What it eliminates	Source
Input longer than 200K tokens	Haiku 4.5	Ch 14
Output longer than 64K tokens	Haiku 4.5	Ch 14
Output longer than 128K (batch)	Fable 5 and Haiku 4.5 — only six models have the 300K beta	Ch 14
Needs task budgets	Everything except Opus 5, Fable 5, Mythos 5, Opus 4.8, Opus 4.7 — **not Sonnet 5**	Ch 13
Needs the `effort` parameter	Haiku 4.5	Ch 13
Needs interleaved thinking with tools	Haiku 4.5	Ch 13
Needs thinking fully disabled	Fable 5; Opus 5 above `high` effort	Ch 13
Needs non-default `temperature`	Everything from Opus 4.7 onward	Ch 13
Knowledge of events after Feb 2025 without retrieval	Haiku 4.5	Ch 14
Data residency in a specific region	Constrains platform and endpoint, not model	Ch 14
Model must survive N months	Check the retirement floor	Ch 14

Chapter 14's preflight validator checks most of these mechanically. Running it across your candidate list before you spend a week labelling is ten seconds well spent, and it is the cheapest step in this chapter.

15.12 Visual Analogies

VISUAL ANALOGY

Hiring, not shopping

Choosing a model resembles hiring more than buying a component. You cannot inspect the internals, the CV is somebody else's benchmark, and the only evidence that means anything is a work sample on your actual task — which is what an evaluation set is. Interviews are noisy, so you use several tasks; and you would not hire the candidate who did marginally better on one exercise if they cost five times as much and the difference was within the noise of your process.

Where it breaks. People improve after you hire them and models do not: parameters are frozen, and Chapter 1's forward-only inference means nothing carries over between requests. The analogy also understates how cheap the trial is. You can run all ten candidates on the same five hundred examples in an afternoon for a few dollars, which is not true of interviewing ten people, and that asymmetry is precisely why there is no excuse for deciding on a leaderboard.

VISUAL ANALOGY

Choosing a courier by price alone

Two couriers: one charges a pound and loses one parcel in five, the other charges three pounds and loses one in fifty. Ranking by price picks the first. Ranking by price-per-delivered-parcel still picks the first. Only when you write down what a lost parcel costs — a refund, a reshipment, a complaint, a customer who does not come back — does the second become obviously cheaper.

Where it breaks. A lost parcel announces itself and a wrong model output frequently does not. Chapter 8's whole subject is that a confident, fluent, wrong answer looks exactly like a right one, so your failure *rate* is an estimate with an interval and your failure *cost* includes the ones you never detected. That pushes the break-even thresholds of section 15.6 lower still, in the direction of buying quality.

VISUAL ANALOGY

A dimmer switch on every bulb

The effort parameter is a dimmer, and the model is which bulb you installed. Comparing four bulbs at full brightness tells you about four points in a two-dimensional space where you could have been standing anywhere. Dimming the bright bulb often gives you more light per watt than running the dim bulb at full power — which is exactly what `sonnet-5/high` beating `opus-5/low` means.

Where it breaks. A dimmer is linear and monotone; effort is neither. Chapter 13 established diminishing and occasionally *negative* returns, and section 15.24 shows `opus-5/xhigh` scoring below `opus-5/high` at $14,000 a month more. Effort is a behavioural signal rather than a dial with a guaranteed direction, so every setting has to be measured rather than interpolated.

15.13 Architecture: Selection Is a Pipeline, Not a Meeting

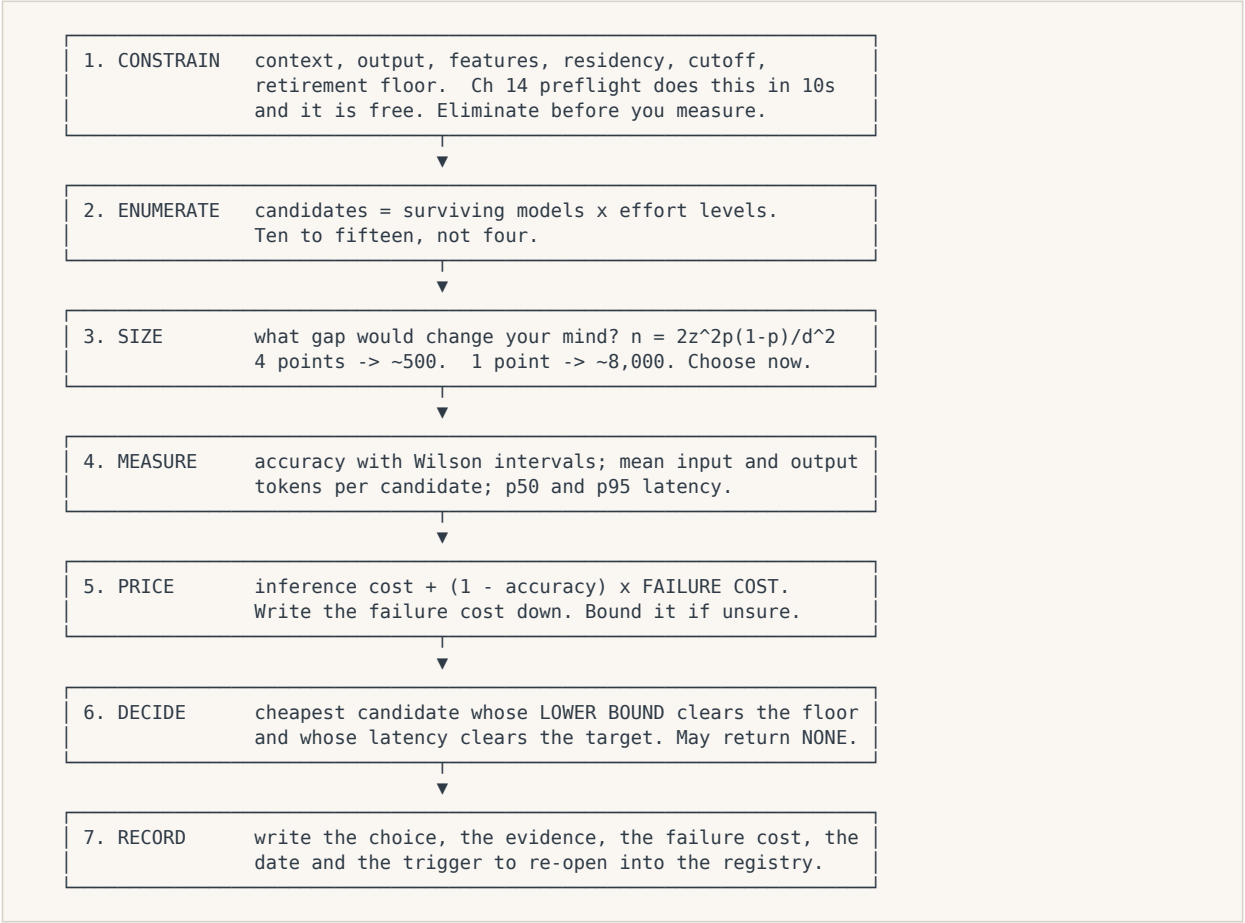


Figure 15.1 — The selection procedure, in the order that saves the most work.

Two design points about that order.

Constraints come first because they are free. Eliminating Haiku 4.5 because your inputs are 400K tokens takes ten seconds and saves you labelling it. Discovering it after a week of evaluation is a week.

The output is a record, not a model name. Chapter 14's registry is where it goes: the chosen candidate, the evidence, the assumed failure cost, the date, and the condition that would re-open the question. A decision without a recorded trigger is a decision nobody revisits, and section 15.14 is about when to revisit.

15.14 When to Re-open the Decision

Model selection is not a one-time event, and the useful discipline is to name the triggers in advance so that re-opening is routine rather than political.

A new model launches. Re-run the evaluation set. This is cheap by construction if step 7 was done properly, and Chapter 14's registry makes applying the result one line.

A model you use is deprecated. Forced, and on somebody else's clock. Chapter 14's lifecycle table is the calendar.

Your traffic distribution shifts. The evaluation set was sampled from a distribution. If the distribution moved, the evidence is stale even though nothing about the models changed.

Your failure cost changes. A new SLA, a new regulatory exposure, a human review step added or removed. Section 15.6's break-even thresholds move, and they move the answer.

Your volume changes by an order of magnitude. At a thousand requests a month the difference between candidates is noise on an expenses claim. At ten million it funds a team.

A pinned model drifts. Chapter 14's serving infrastructure can move under a stable ID. Your evaluation set is the only instrument that would notice, which is the argument for re-running it on a schedule rather than only on demand.

15.15 Code Walkthrough: Selection as Arithmetic

The harness is two modules and twenty-six tests. Its central object is the one section 15.3 argued for.

```
@dataclass
class Candidate:
    model: str
    effort: object = None
    successes: int = 0
    trials: int = 0
    input_tokens: int = 4_000
    base_output_tokens: int = 400
```

A candidate carries its own evidence — `successes` out of `trials` — rather than a bare accuracy, which is what makes every interval in the chapter computable rather than remembered. It carries its token profile too, because effort changes what you generate rather than what you are charged per token:

```
EFFORT_OUTPUT_MULTIPLIER = {
    None: 1.0, "low": 0.6, "medium": 1.0,
    "high": 1.8, "xhigh": 3.2, "max": 5.0,
}
```

Those multipliers are a *model* of the relationship, not a measurement of it, and the docstring says so. Measure your own; the shape — that effort moves output tokens by roughly 5× across the range — is what transfers, and it is why `opus-5/xhigh` costs $52,000 a month where `opus-5/low` costs $26,000 on the same model at the same rate.

`cost()` delegates to Chapter 14's registry, so there is exactly one rate card in the book:

```
def cost(self, requests=1, **kw):
    import migrate
    return migrate.cost(self.model, self.input_tokens,
                        self.output_tokens, requests, **kw)
```

`established_better` **is a two-proportion z-test, and it is allowed to say no.**

```
def established_better(a, b, z=1.96):
    if a.trials == 0 or b.trials == 0:
        return False
```

```
    p1, p2 = a.accuracy, b.accuracy
    if p1 <= p2:
        return False
    n1, n2 = a.trials, b.trials
    p = (a.successes + b.successes) / (n1 + n2)
    se = np.sqrt(p * (1 - p) * (1 / n1 + 1 / n2))
    return (p1 - p2) / se > z if se else False
```

The pooled standard error is the standard construction; the important line is the early `return False` when `p1 <= p2`, which makes the relation directional. "Not established" is the frequent answer and the whole point of the module.

The two frontiers differ by one word, and it is the word that matters.

```
# economic: is anything cheaper AND better?
if (a.accuracy >= b.accuracy and a.cost(r) <= b.cost(r)
        and (a.accuracy > b.accuracy or a.cost(r) < b.cost(r))):
    dominated = True
# defensible: is b ESTABLISHED better than everything cheaper?
cheaper = [a for a in cands if a.cost(r) < b.cost(r)]
if all(established_better(b, a) for a in cheaper):
    out.append(b)
```

The economic test uses point estimates and answers "could I do better for less?". The defensible test uses evidence and answers "can I justify paying more?". They are different questions and they produce different sets, which is why the harness computes both.

`decide()` reads the lower bound, not the point estimate.

```
ok = [c for c in cands if c.interval(z)[0] >= quality_floor]
if budget is not None:
    ok = [c for c in ok if c.cost(requests) <= budget]
if not ok:
    return None
return min(ok, key=lambda c: c.cost(requests))
```

Three properties fall out of five lines. It can return `None`, and `None` is an answer. It never picks a dearer candidate when a cheaper one clears the bar. And a candidate that scored 0.85 on twenty examples cannot clear an 0.80 floor, because its lower bound is 0.64 — there is a test asserting exactly that, and it is the test that would have stopped the introduction's eleven-minute meeting.

`naive_decide()` is included deliberately.

```
def naive_decide(cands):
    return max(cands, key=lambda c: c.accuracy)
```

One line, picking the top point estimate. It is not a straw man; it is what a table sorted descending invites, and section 15.22 measures what it costs.

15.16 Mini Project: The Selection Harness

Goal. Turn "which model should we use" into a computation whose inputs are an evaluation set, a rate card and a failure cost, and whose output is a defensible choice — then measure how often the computation beats the intuition it replaces.

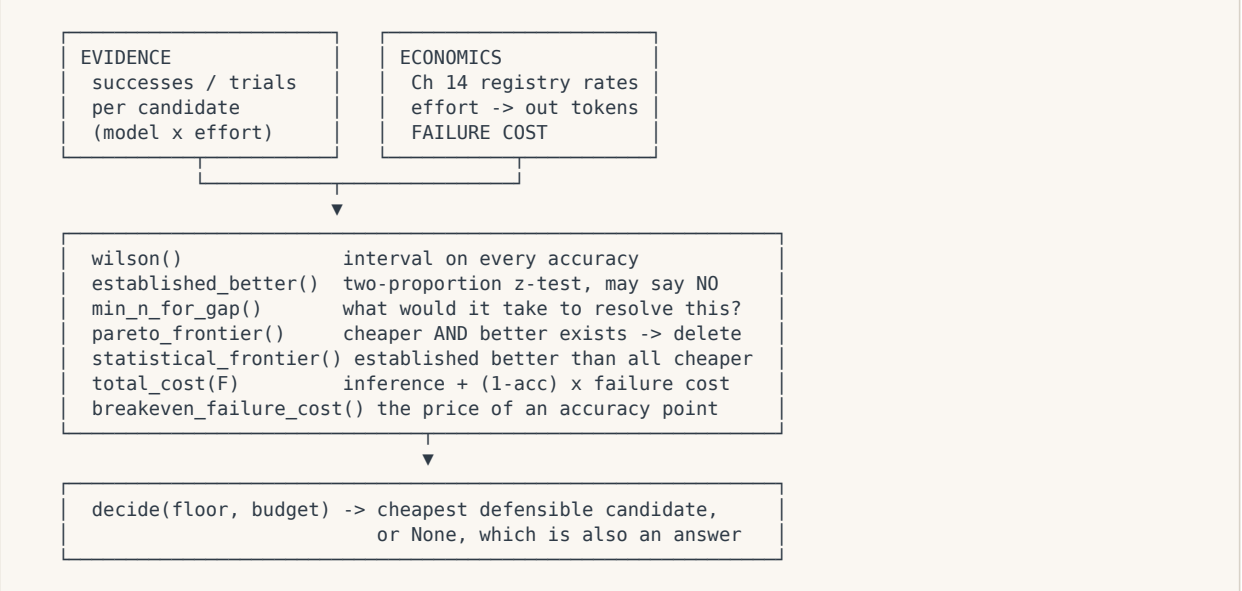


Figure 15.2 — The harness. Evidence in, a defensible choice out.

```
ch15/
  choose.py          Candidate, intervals, frontiers, costs, decide()
  registry.py        copied from ch14 — the rate card
  migrate.py         copied from ch14 — cost()
  report.py          the seven panels below
  test_choose.py     26 tests, 0.36 seconds
```

15.17 The Simulation, and Why It Is One

The panels below simulate an evaluation rather than calling an API. That is a deliberate choice with three reasons, and it is worth stating them because a simulation used carelessly is exactly the sort of thing this book complains about.

Reproducibility. The book must run offline, from a fixed seed, with no key and no bill.

We can score the decision instead of the model. In a real evaluation you never learn the true accuracy, so you can never tell whether your rule chose well — only what it chose. Here the ground truth is ours, so the harness can ask the question that matters: *how often does this rule pick badly, and what does that cost?*

We can run the protocol two thousand times. One evaluation tells you one story. A rule's error rate is a property of the rule, and no single evaluation can measure it.

The ground-truth accuracies below are invented and the arithmetic done to them is not. It is the arithmetic that transfers.

```
TRUTH = {
    ("claude-haiku-4-5-20251001", None): 0.720,
    ("claude-sonnet-5", "low"):    0.795,
    ("claude-sonnet-5", "medium"): 0.845,
```

```
    ("claude-sonnet-5", "high"):   0.862,
    ("claude-opus-5", "low"):      0.851,
    ("claude-opus-5", "medium"):   0.878,
    ("claude-opus-5", "high"):     0.890,
    ("claude-opus-5", "xhigh"):    0.894,
    ("claude-fable-5", "medium"):  0.897,
    ("claude-fable-5", "high"):    0.903,
}
```

Ten candidates from four models. The evaluator never sees this dictionary.

15.18 Panel A — Ranked by Accuracy

Twenty thousand examples per candidate, so the intervals are tight and the panel is about ordering rather than uncertainty.

```
A  TEN CANDIDATES, RANKED BY ACCURACY (n=20,000 each)
             candidate     acc             95% CI  out tok    $/month
          fable-5/high  0.9050 [0.9009,0.9090]      720     76,000
        fable-5/medium  0.8980 [0.8937,0.9021]      400     60,000
           opus-5/high  0.8941 [0.8897,0.8982]      720     38,000
          opus-5/xhigh  0.8927 [0.8883,0.8969]     1280     52,000
         opus-5/medium  0.8790 [0.8744,0.8834]      400     30,000
         sonnet-5/high  0.8588 [0.8539,0.8636]      720     15,200
            opus-5/low  0.8529 [0.8479,0.8577]      240     26,000
       sonnet-5/medium  0.8417 [0.8365,0.8466]      400     12,000
          sonnet-5/low  0.7924 [0.7868,0.7980]      240     10,400
    haiku-4-5-20251001  0.7192 [0.7130,0.7254]      400      6,000
```

Read top-down and you buy `fable-5/high` for $76,000 a month. The gap from there to `sonnet-5/medium` is **6.3 accuracy points and $64,000**, which is roughly $10,000 a month per point — a number worth having in your head before anybody says "we should use the best model".

Two rows are already interesting before any further analysis. `opus-5/xhigh` sits *below* `opus-5/high` while costing $14,000 more, which is Chapter 13's diminishing returns going negative. And `opus-5/low` sits *below* `sonnet-5/high` while costing $10,800 more, which is the effort axis crossing the model axis.

15.19 Panel C — The Two Frontiers

```
C  THE ECONOMIC FRONTIER
  on the frontier (8):
      haiku-4-5-20251001  0.7192      6,000
            sonnet-5/low  0.7924     10,400
         sonnet-5/medium  0.8417     12,000
           sonnet-5/high  0.8588     15,200
           opus-5/medium  0.8790     30,000
             opus-5/high  0.8941     38,000
          fable-5/medium  0.8980     60,000
            fable-5/high  0.9050     76,000
  dominated (2) — cheaper AND better exists:
              opus-5/low beaten by sonnet-5/high
            opus-5/xhigh beaten by opus-5/high
  DEFENSIBLE frontier — established better than everything
  cheaper (7 of 10):
```

```
      haiku-4-5-20251001  0.7192      6,000
            sonnet-5/low  0.7924     10,400
         sonnet-5/medium  0.8417     12,000
           sonnet-5/high  0.8588     15,200
           opus-5/medium  0.8790     30,000
             opus-5/high  0.8941     38,000
            fable-5/high  0.9050     76,000
```

Two candidates leave on the economic test and require no discussion: something is cheaper *and* better, so there is nothing to weigh.

The defensible frontier removes one more, and it is the expensive one. `fable-5/medium` **at $60,000 a month is not established better than** `opus-5/high` **at $38,000**, even on twenty thousand examples — the point-estimate gap is 0.4 of a point and the test cannot resolve it. Under the defensible reading you cannot justify the extra $22,000 a month, because you cannot demonstrate what it bought.

Note the asymmetry that makes this useful. The economic frontier can only ever shrink your options by finding something strictly better. The defensible frontier shrinks them by finding something you cannot prove, which is the failure mode that costs money rather than quality.

15.20 Panel B — Cost per Correct Answer

```
B  THE SAME TEN, RANKED BY COST PER CORRECT ANSWER
             candidate     acc    $/month  $/correct/M
    haiku-4-5-20251001  0.7192      6,000        8,342
          sonnet-5/low  0.7924     10,400       13,124
       sonnet-5/medium  0.8417     12,000       14,258
         sonnet-5/high  0.8588     15,200       17,699
            opus-5/low  0.8529     26,000       30,484
         opus-5/medium  0.8790     30,000       34,132
           opus-5/high  0.8941     38,000       42,503
          opus-5/xhigh  0.8927     52,000       58,254
        fable-5/medium  0.8980     60,000       66,815
          fable-5/high  0.9050     76,000       83,978
```

The order is close to an exact inversion of Panel A. Accuracy ranks the models; cost per correct answer ranks the decisions — and produces the opposite recommendation from the same evidence, which should be enough on its own to retire the accuracy-sorted slide.

But now the warning from section 15.5, made concrete. This metric does not reward accuracy; it rewards the better *ratio*, and price ratios in this family are larger than accuracy ratios. Haiku 4.5 at **50%** accuracy would still beat Opus 5 at **95%** on cost per correct answer, because the price gap is fivefold and the accuracy gap is twofold. There is a test asserting it:

```
def test_a_five_fold_price_gap_beats_a_two_fold_accuracy_gap():
    cheap = c("claude-haiku-4-5-20251001", k=500, n=1000)
    dear = c("claude-opus-5", "medium", k=950, n=1000)
    assert cheap.cost_per_correct() < dear.cost_per_correct()
```

A metric that prefers a coin flip at half price is a metric with an assumption in it, and the assumption is that a wrong answer is free. Section 15.23 removes it.

15.21 Panel D — One Evaluation of One Hundred

Now the realistic case: the evaluation set a careful team actually builds.

```
D  ONE EVALUATION OF 100 — WHAT IS ACTUALLY ESTABLISHED?
             candidate     acc             95% CI   width
           opus-5/high    0.91 [0.8377,0.9519]  0.1142
        fable-5/medium    0.90 [0.8256,0.9448]  0.1191
          fable-5/high    0.90 [0.8256,0.9448]  0.1191
          opus-5/xhigh    0.89 [0.8137,0.9375]  0.1238
         opus-5/medium    0.87 [0.7902,0.9224]  0.1322
       sonnet-5/medium    0.85 [0.7672,0.9069]  0.1398
           opus-5/low     0.84 [0.7558,0.8990]  0.1433
          sonnet-5/low    0.79 [0.7002,0.8583]  0.1581
         sonnet-5/high    0.79 [0.7002,0.8583]  0.1581
    haiku-4-5-20251001    0.72 [0.6251,0.7986]  0.1735
  point-estimate winner: opus-5/high
  candidates it is ESTABLISHED better than: 3 of 9
  cheapest candidate whose LOWER bound clears 0.80: opus-5/high
  to resolve a 0.04 gap around 0.88: 508 examples per arm
  to resolve a 0.02 gap around 0.88: 2,029 examples per arm
  to resolve a 0.01 gap around 0.88: 8,114 examples per arm
```

Four things to notice, and the fourth is the one that changes practice.

The intervals are eleven to seventeen points wide. A hundred examples buys you an answer of the form "somewhere between 0.84 and 0.95", which does not distinguish between candidates separated by one or two points.

The winner is established better than three of nine. It genuinely beats `haiku-4-5`, `sonnet-5/low` and `sonnet-5/high` — which, note, are the *cheap* ones. It is not established better than any of the five candidates near it in price.

The true ordering has been scrambled. `sonnet-5/high` (true accuracy 0.862) drew 0.79 and landed at the bottom, below `sonnet-5/low` (true 0.795) and tied with it. On this sample, an entirely reasonable team would have concluded that raising Sonnet 5's effort does nothing. It does; the evaluation was too small to see it.

The floor rule buys the most expensive thing on the list. This is the counter-intuitive result. Applying "cheapest candidate whose lower bound clears 0.80" to this sample returns `opus-5/high`, because it is the *only* candidate whose lower bound clears 0.80 at all. The disciplined rule and the naive rule agree here — and they agree on the expensive answer, because **a small evaluation cannot defend a cheap choice.** Discipline does not save you money on a hundred examples. It saves you money on two thousand, and section 15.22 measures how much.

15.22 Panel E — Two Thousand Replications

Run the whole protocol two thousand times at four evaluation sizes, and score the decisions against the ground truth the evaluator never sees.

```
E  TWO THOUSAND REPLICATIONS: HOW OFTEN IS EACH RULE WRONG?
       n  naive picks  naive extra  floor rule  floor cost  no choice
      50       0.0179       45,675      0.0271      35,172      6.3%
```

```
     100       0.0125       51,298      0.0390      23,172      0.1%
     400       0.0061       60,341      0.0528      13,443      0.0%
    2000       0.0030       67,929      0.0582      11,993      0.0%
```

The two "picks"/"rule" columns are the mean shortfall in *true* accuracy against the best available candidate; the cost columns are the mean monthly bill of the chosen candidate. Three readings.

The naive rule gets more accurate and more expensive as the evaluation grows. Its accuracy shortfall falls from 0.0179 to 0.0030 while its bill rises from $45,675 to $67,929. That is exactly what "pick the highest point estimate" converges to: with enough data it reliably identifies the genuinely best candidate, which is the most expensive one, and pays for it. The rule works perfectly and answers the wrong question.

The floor rule goes the other way. Its accuracy shortfall *rises* from 0.0271 to 0.0582 while its bill falls from $35,172 to $11,993. It is deliberately giving up 5.8 points of accuracy against the best candidate — because the floor said 0.80 and it does not pay for quality above the bar it was given.

The difference between those two columns is the whole chapter. At two thousand examples the naive rule pays $67,929 for 0.890-ish accuracy and the floor rule pays $11,993 for 0.845-ish. Whether that trade is right depends entirely on your floor and your failure cost, which is section 15.23 — but the trade is now visible and priced, rather than made by accident by whoever sorted the table.

KEY TAKEAWAYS

What an evaluation set is worth, in money

The floor rule's bill falls from **$35,172** at fifty examples to **$11,993** at two thousand. Nothing about the models changed and no accuracy improved. What changed is what could be *proved*, and what could be proved is what you were allowed to stop paying for.

$23,179 a month is the measured value of a 2,000-example evaluation set over a 50-example one, on this workload. Labelling 1,950 more examples is days of work. This is the argument for evaluation stated in the only unit that reliably wins a budget meeting.

One more column deserves attention. At fifty examples the floor rule returns **no choice 6.3% of the time** — no candidate's lower bound cleared 0.80. That is not a failure of the rule; it is the rule working. The right response to "not established" is a bigger evaluation set, not a bigger model.

15.23 Panel G — What Does a Wrong Answer Cost You?

Total monthly cost including the cost of being wrong, at five prices per failure.

```
G  WHAT DOES A WRONG ANSWER COST YOU?
             candidate       F=$0    F=$0.05     F=$0.2       F=$1       F=$5
    haiku-4-5-20251001      6,000     20,038     62,150    286,750  1,409,750
          sonnet-5/low     10,400     20,778     51,910    217,950  1,048,150
       sonnet-5/medium     12,000     19,917     43,670    170,350    803,750
         sonnet-5/high     15,200     22,260     43,440    156,400    721,200
            opus-5/low     26,000     33,355     55,420    173,100    761,500
         opus-5/medium     30,000     36,052     54,210    151,050    635,250
           opus-5/high     38,000     43,298     59,190    143,950    567,750
          opus-5/xhigh     52,000     57,367     73,470    159,350    588,750
        fable-5/medium     60,000     65,100     80,400    162,000    570,000
```

```
          fable-5/high     76,000     80,750     95,000    171,000    551,000
```

Read across the columns and the winner walks down the table:

If a wrong answer costs	The cheapest total-cost candidate is
$0	`haiku-4-5-20251001` — $6,000
$0.05	`sonnet-5/medium` — $19,917
$0.20	`sonnet-5/high` — $43,440
$1.00	`opus-5/high` — $143,950
$5.00	`fable-5/high` — $551,000

Four different answers from one set of measurements. The accuracy figures did not change, the rate card did not change, and the recommendation moved from the cheapest model in the family to the most expensive one — driven entirely by a number that was never on the slide.

The thresholds, computed directly:

```
         against haiku-4-5  breakeven F
           sonnet-5/medium        0.049
             sonnet-5/high        0.066
               opus-5/high        0.183
              fable-5/high        0.377
```

If a wrong answer costs more than **4.9 pence** to deal with, Sonnet 5 at medium effort is not merely better than Haiku 4.5 — it is **cheaper overall**. Above **18.3 pence**, so is Opus 5 at high effort. Put a human anywhere near the failure path and every one of those thresholds is cleared by an order of magnitude: four minutes of review at £40 an hour is £2.67, which is fourteen times the Opus 5 threshold and seven times the Fable 5 one.

> WARNING
>
> **The default assumption is zero, and it is never zero**
>
> A table that ranks candidates by cost, or by cost per correct answer, has silently assumed a failure costs nothing. That assumption is almost always false and it always favours the cheap model. Write the number down. If you cannot estimate it precisely, bound it — "somewhere between 10p and £3" is enough to settle most of these comparisons, because the thresholds are pennies.

15.24 Panel F — The Effort Axis Versus the Model Axis

```
F  THE EFFORT AXIS VERSUS THE MODEL AXIS
           cheaper           dearer  acc gain   extra $    $/point  est?
     sonnet-5/high       opus-5/low   -0.0059   +10,800         --    no
       opus-5/high     opus-5/xhigh   -0.0014   +14,000         --    no
       opus-5/high   fable-5/medium   +0.0040   +22,000     55,696    no
   sonnet-5/medium       opus-5/low   +0.0112   +14,000     12,444   yes
```

Row by row, because each one is a different lesson.

Row 1: switching to a bigger model at low effort is worse than raising effort on the smaller one. `sonnet-5/high` beats `opus-5/low` on accuracy *and* costs $10,800 a month less. This is the documentation's claim — *tuning effort is often a better lever than switching models* — as a measurement rather than an assertion, and it is invisible to any comparison that holds effort fixed.

Row 2: more effort is not more accuracy. `opus-5/xhigh` scores below `opus-5/high` at $14,000 a month more, on the same model. Chapter 13's diminishing returns went negative, and Chapter 13's rule about running an effort sweep rather than assuming monotonicity is why you would find out.

Row 3: a gap you cannot establish on twenty thousand examples. `fable-5/medium` is 0.4 of a point ahead of `opus-5/high` for $22,000 a month — $55,696 per accuracy point — and the two-proportion test says **not established** even at n=20,000. Section 15.9's arithmetic explains why: resolving a 0.4-point gap needs roughly 50,000 examples per arm. The correct conclusion is not "get more data". It is that a difference you cannot demonstrate on twenty thousand examples is a difference you should stop paying for.

Row 4: a real improvement, priced. `opus-5/low` is 1.1 points ahead of `sonnet-5/medium` for $14,000 a month — $12,444 per point — and it *is* established. This is what a defensible upgrade looks like: a measurable gain with a stated price, which you can then compare against your failure cost from section 15.23.

Two of four rows have the dearer option performing worse. One has a gain nobody can prove. Comparing four models at their default effort would have surfaced none of it.

15.25 Evaluation and Deployment

Run `python3 report.py` (a few seconds, no network) and `python3 -m pytest` (26 tests, 0.36 seconds). To adopt it:

1. **Replace `TRUTH` with a real runner** that calls each candidate on your evaluation set and records successes and trials. Everything downstream is unchanged.
2. **Record real token counts per candidate** rather than the effort multipliers. Read `usage.output_tokens` and, from Chapter 13, `usage.output_tokens_details.thinking_tokens`.
3. **Write down your failure cost**, or a range. This is a conversation with operations and finance, not with engineering, and it is the highest-leverage half-hour in the process.
4. **Run Chapter 14's `preflight()` over your candidate list first**, so you do not label examples for a candidate that will 400.
5. **Store the decision in the registry**: the chosen candidate, the evidence, the assumed failure cost, the date and the trigger to re-open.
6. **Re-run on a schedule**, not only on demand. It is the same instrument that detects Chapter 14's serving-infrastructure drift.

15.26 Hands-on Labs

Lab 1 — Enumerate honestly. List every candidate for one workload you own: every model that clears your constraints, times every effort level it supports. *Success criterion:* a list of at least eight candidates, and an explanation for every model you excluded.

Lab 2 — Size before you label. Decide what accuracy difference would change your decision, then compute the examples per arm needed to resolve it. *Success criterion:* a number, and a written decision to either build that many or to accept that smaller differences will not be resolved.

Lab 3 — Write down the failure cost. With whoever owns the downstream process, estimate what one wrong answer costs. *Success criterion:* a range, with the components named — retry, review time, contact rate, refund probability.

Lab 4 — Compute your break-evens. For your two most plausible candidates, compute `F*` and compare it against Lab 3's range. *Success criterion:* you can say which candidate is cheaper *overall* and at what failure cost that flips.

Lab 5 — Run both rules. Apply `naive_decide` and `decide` to your measurements. *Success criterion:* you know whether they agree, and if they disagree, the monthly cost of the difference.

15.27 Lab Solutions

Lab 1. The commonest omission is effort levels, and the second commonest is the Batch API, which halves every price and belongs as a separate candidate whenever latency permits. Chapter 14's `preflight()` mechanically excludes candidates that would 400.

Lab 2. `n ≈ 2z²p(1−p)/δ²`. Around 88% accuracy: 508 examples per arm for a four-point gap, 2,029 for two points, 8,114 for one. If the honest answer is "one point would change our decision", the honest follow-up is "then we need eight thousand examples", and the usual outcome of saying that out loud is that one point stops changing the decision.

Lab 3. Decompose it: probability the failure is detected, cost of a retry, probability it reaches a human, minutes of review, loaded hourly cost, probability it reaches a customer, expected cost of that. Multiply through. The number is usually larger than people guess, because the undetected failures — Chapter 8's confident, fluent, wrong answers — carry the most weight and are the easiest to forget.

Lab 4. `F* = (cost_d − cost_c) / (acc_d − acc_c)`. Compare against Lab 3's range. If your range sits entirely above `F*`, buy the dearer candidate and stop arguing; if it sits entirely below, buy the cheaper one. If it straddles `F*`, tightening the failure-cost estimate is worth more than tightening the accuracy estimate.

Lab 5. They will agree on a small evaluation and diverge on a large one, for the reason Panel D and Panel E showed together: discipline only saves money once the intervals are narrow enough to defend a cheaper choice. If they disagree, the disagreement is a monthly figure, and that figure is what a bigger evaluation set is worth to you.

15.28 Best Practices

BEST PRACTICE

Choosing a model without fooling yourself

1. **Enumerate candidates, not models.** Every model times every effort level, plus batch where latency allows.
2. **Apply constraints first.** They are free and they eliminate candidates before you label anything.
3. **Build the evaluation set from your own traffic**, including the hard slices, with the metric chosen before you see scores.
4. **Size it for the difference that would change your mind**, and accept that smaller differences will go unresolved.
5. **Print intervals on every number.** A point estimate is not a finding.
6. **Write down the failure cost.** A bounded estimate beats the implicit zero that every cost table assumes.
7. **Compute break-even failure costs**, not just costs. They convert model arguments into operations questions.
8. **Delete dominated candidates without discussion.** Cheaper and better is not a trade-off.
9. **Apply the floor rule**: cheapest candidate whose lower bound clears the bar.
10. **Let the answer be "not established".** It means measure more, not spend more.
11. **Sweep effort on every model you shortlist.** Settings do not transfer, and returns are not monotone.
12. **Consider batch before you consider downgrading.** 50% off both directions moves the whole frontier.
13. **Record the decision, the evidence and the re-open trigger** in Chapter 14's registry.
14. **Re-run on a schedule.** New models, deprecations, traffic shift, failure-cost changes, and infrastructure drift all invalidate the evidence without touching your code.

15.29 Common Mistakes

COMMON MISTAKE

Deciding from a leaderboard

Symptom. A model is chosen from a published benchmark table, and production quality does not match. **Cause.** Chapter 12's five traps: contamination, saturation, scaffolding, prompt sensitivity, missing intervals. A leaderboard measures somebody else's distribution with somebody else's harness. **Fix.** Build the evaluation set. The documentation's own guidance says it is the most important step in the process. **Detection.** Ask what the benchmark's `n` was and whether intervals were published. If nobody knows, the table has not established anything.

COMMON MISTAKE

Comparing models at fixed effort

Symptom. A shortlist of four models, all at `high`, and a conclusion that the expensive one is needed. **Cause.** Treating effort as a tuning detail rather than an axis. `high` is the default and the most expensive setting. **Fix.** Enumerate (model, effort) pairs. Panel F found two candidates where the dearer option is *less* accurate, both invisible at fixed effort. **Detection.** Count your candidates. Four is a symptom.

COMMON MISTAKE

Selecting on point estimates

Symptom. "Model A scored 91%, model B scored 90%, so we chose A" — for six times the price. **Cause.** No intervals, and no sizing step. On a hundred examples a one-point difference is not measurable. **Fix.** Print Wilson intervals; use `established_better`; apply the floor rule. Section 15.22 measures the naive rule converging on the most expensive candidate. **Detection.** Compare the gap you acted on against the interval width. If the gap is smaller, you acted on noise.

COMMON MISTAKE

Assuming a wrong answer is free

Symptom. A cost-driven choice of the cheapest model, followed by a support or review workload nobody costed. **Cause.** Ranking by cost, or by cost per correct answer, both of which embed a zero failure cost. **Fix.** `total_cost = inference + (1 − accuracy) × failure_cost`. Section 15.23's break-evens are 5p to 38p; almost any human-in-the-loop process clears them. **Detection.** Ask what happens to a wrong answer. If the answer involves a person, the failure cost is pounds, not pence.

COMMON MISTAKE

Treating a small evaluation as cheap

Symptom. A hundred-example evaluation, and a conclusion that only the top model meets the quality bar. **Cause.** Wide intervals mean only high scorers have a defensible lower bound, so the disciplined rule buys the expensive candidate. Section 15.21 shows exactly this. **Fix.** Label more. Section 15.22 prices the difference at $23,179 a month between fifty examples and two thousand. **Detection.** If your interval width exceeds the gaps between your candidates, your evaluation cannot rank them.

COMMON MISTAKE

Chasing a difference nobody can establish

Symptom. Sustained spend on a top-tier model for a fraction of a point. **Cause.** A point estimate that is genuinely higher and a gap that is genuinely unresolvable. `fable-5/medium` versus `opus-5/high`: 0.4 points, $22,000 a month, not established at n=20,000. **Fix.** Compute `min_n_for_gap`. If the required sample is implausible, the difference is not actionable and should not be purchased. **Detection.** Divide the cost difference by the accuracy difference. If the answer is tens of thousands per point and the gap is not established, stop.

COMMON MISTAKE

Deciding once

Symptom. A model chosen in 2024 still in production, with nobody able to say why. **Cause.** No recorded decision, no recorded evidence, no re-open trigger. **Fix.** Section 15.14's trigger list, and Chapter 14's registry as the place the record lives. **Detection.** Ask why you use the model you use. If the answer is "it is what we started with", the decision has expired.

15.30 Performance Considerations

Latency belongs in the frontier as a third axis, and the honest treatment is a constraint rather than an objective: candidates that miss the p95 target are eliminated, and the rest compete on cost. That keeps the arithmetic two-dimensional and stops a latency preference from quietly outranking a quality floor.

Three things move latency more than the model choice does. **Effort**, which changes the output token count by up to 5× and therefore moves the dominant term of Chapter 11's model directly. **Fast mode**, up to 2.5× higher output speed on Opus 5 and Opus 4.8 at premium pricing, and the only lever that buys speed without cutting tokens. And **thinking**, which Chapter 13 showed can multiply the decode term seventeen-fold on a short answer.

The reframing that changes decisions: *"we cannot afford the top model"* and *"we cannot wait"* are different sentences. If the workload tolerates delay, the Batch API's 50% discount on both directions frequently makes the top candidate cheaper than the mid-tier synchronous one, and the correct answer is a queue rather than a downgrade. Chapter 30 builds it.

15.31 Security Considerations

The evaluation set is production data. It is sampled from real traffic, which means it inherits every classification, retention and residency obligation that traffic carries, and it will be copied to laptops and pasted into notebooks unless you prevent it. Treat it as the sensitive dataset it is; Chapter 61 covers the obligations.

A cheaper model is a different safety posture, not just a lower score. Refusal behaviour, susceptibility to injected instructions and adversarial robustness vary by model and are not summarised by an accuracy figure on your happy path. Any candidate that reaches a shortlist should be run against an adversarial set as well as a quality set. Chapter 58 builds that set.

Failure cost includes the tail, and the tail is where the security cost lives. Most wrong answers cost a retry. A small number cost a data disclosure, a wrong payment or a regulatory notification. If your failure distribution has a tail like that, the mean failure cost in section 15.23 understates the case for quality, and the right instrument is a floor on the tail rather than an average.

Do not let the selection procedure leak. A published statement that you route class X to a cheap model and class Y to an expensive one tells an attacker exactly how to get their input classified as X. Routing rules are part of your attack surface; Chapter 50 treats them as such.

15.32 Real Production Examples

Classification at volume. The archetype for efficiency-first. Start with Haiku 4.5, measure with intervals, and let section 15.23's break-even decide whether to step up. The trap is the implicit zero failure cost: if misclassifications reach a human, Sonnet 5 at medium effort is usually cheaper overall despite costing twice as much per request.

Agentic coding. The archetype for capability-first, and the case where the documentation, Chapter 13 and this chapter all agree. Failure cost is high (a bad patch costs review time or an incident), the verifier is cheap (Chapter 13's tests), and the work is long-horizon, which is what `xhigh` effort and task budgets

exist for. Note that task budgets rule out Sonnet 5, which is a constraint rather than a preference.

Retrieval-grounded question answering. The hard case, because Chapter 8 established that the natural cheap verifier is unsound in exactly the dangerous direction and Chapter 13 showed that test-time compute cannot rescue it. Failure cost is high and hard to measure, because a confident fabrication may never be detected. When the failure rate itself is uncertain, buy quality: the asymmetry favours it.

Subagents in an orchestrator topology. The documentation names `low` effort on subagents as a standard pattern, and the selection question is per-role rather than per-system. An orchestrator on Opus 5 at high effort with subagents on Haiku 4.5 is a fleet, and Chapter 14's `ROLES` map is where the answers live. Chapter 50 builds the topology.

15.33 Certification Notes

Concept	Exam domain	What is likely to be asked
The four criteria	D1 Agentic Architecture & Orchestration	Capabilities, speed, cost, effort
Efficiency-first vs capability-first	D1	Which strategy suits a described scenario
Effort as a lever	D4 Prompt Engineering & Structured Output	Tuning effort is often better than switching models
The selection matrix	D1	Which model to *start with* for a use case
Evaluation sets	D5 Context Management & Reliability	The most important step; your task, your metric, private
Intervals and establishment	D5	Overlapping intervals mean nothing has been established
Sample sizing	D5	Halving the gap quadruples the sample
Cost per correct answer	D5	When it is right, and the failure-cost assumption inside it
Dominated candidates	D1	Cheaper and better means delete, not trade off
Fast mode	D1, D5	Opus 5 and Opus 4.8, up to 2.5× output speed, premium pricing, no batch
Batch as a selection lever	D5	50% off both directions moves the frontier
Constraints that eliminate	D1, D5	Context, output, task budgets, effort support, cutoff, residency

CERTIFICATION NOTE

Exam Tips

- **The four criteria are capabilities, speed, cost and effort.** Effort is the one candidates forget.
- **"Tuning effort is often a better lever than switching models"** is close to a direct quote. Expect it as a correct option.
- **Efficiency-first starts with Haiku 4.5; capability-first starts with Opus 5.** Match the strategy to the scenario's emphasis.
- **The evaluation set is described as the most important step in the process.** Any option preferring a published benchmark is wrong.
- **Fast mode: Opus 5 and Opus 4.8 only, up to 2.5×, premium pricing, not with Batch.**
- **If a question gives two scores and no sample size**, the answer is that nothing has been established.
- **If a question describes flat accuracy and rising cost**, look for the dominated candidate.
- **A cheaper model plus a human review step is often dearer overall.** Watch for questions that supply a failure cost — it is supplied because it decides the answer.

15.34 Twenty Practice Questions

1. What three considerations does the documentation say a model choice balances, and what fourth does it add?
2. What is the documented efficiency-first strategy, and when is it best?
3. What is the documented capability-first strategy, and when is it best?
4. Why is a candidate a (model, effort) pair rather than a model?
5. What does the documentation say is the most important step in deciding whether to change models?
6. Two models score 91% and 90% on 100 examples. What have you established?
7. How many examples per arm resolve a two-point gap around 88%?
8. What happens to the required sample when you halve the gap you want to resolve?
9. When is cost per correct answer the right metric, and when is it wrong?
10. Write the formula for total cost including failure cost.
11. Write the formula for break-even failure cost, and say what it means.
12. Define a dominated candidate. What should you do with one?
13. What is the difference between the economic frontier and the defensible frontier?
14. State the floor rule. Why the lower bound rather than the point estimate?
15. Why does a small evaluation set force you to buy an expensive model?
16. What does it mean when the floor rule returns nothing?
17. Which models support fast mode, what does it buy, and what does it cost?
18. Why can a faster model be slower end-to-end than a slower one?
19. Name four constraints that eliminate a candidate before any measurement.
20. Name four triggers for re-opening a model decision.

15.35 Answers and Explanations

1. **Capabilities, speed and cost**, plus **effort** — which the documentation describes as trading intelligence for latency and cost *within* a single model, and often a better lever than switching models. *Omitting effort is the common error and it halves the search space.*
2. **Start with Claude Haiku 4.5**, test thoroughly, evaluate against requirements, upgrade only for specific capability gaps. Best for prototyping, tight latency requirements, cost-sensitive implementations and high-volume straightforward tasks. *It is a documented strategy, not a shortcut.*
3. **Start with Claude Opus 5**, optimise prompts for it, evaluate, then increase efficiency by lowering effort or downgrading over time. Best for complex reasoning, scientific and mathematical work, nuanced understanding, accuracy-over-cost applications, and advanced coding and high-autonomy agentic work. *Note the direction of travel: optimise down, not up.*
4. Because effort changes accuracy, cost and latency **within** a model, and the two axes cross. In section 15.24, `sonnet-5/high` beats `opus-5/low` on accuracy while costing $10,800 a month less. *Comparing models at fixed effort searches a line through a plane.*
5. **Creating benchmark tests specific to your use case** — the documentation says having a good evaluation set is the most important step. *Not choosing the right starting model, and not reading a leaderboard.*
6. **Almost nothing.** On 100 examples the 95% interval is roughly eleven points wide, so the two intervals overlap heavily. A one-point difference at that sample size is indistinguishable from noise. *"A is better" is the wrong answer; "not established" is the right one.*
7. About **2,029** per arm, from $n \approx 2z^2p(1-p)/\delta^2$. Around 90% it is about 1,729, which is Chapter 12's published figure. *A hundred examples resolves roughly a ten-point gap and nothing finer.*
8. It **quadruples**, because n scales as $1/\delta^2$. *This is the single most useful fact in evaluation design, and it argues for deciding that small differences do not matter.*
9. It is right when **a wrong answer is free to discard** — you have a cheap verifier and can retry. It is wrong whenever a failure costs a retry, a review or a customer, because then failures belong in the sum rather than the denominator. *It also does not simply reward accuracy: Haiku 4.5 at 50% beats Opus 5 at 95% on this metric, because the price ratio exceeds the accuracy ratio.*
10. `total = inference_cost + (1 − accuracy) × failure_cost`**.** *The implicit assumption in every accuracy table is that the second term is zero.*
11. `F* = (cost_d − cost_c) / (acc_d − acc_c)` — the failure cost at which the dearer candidate becomes cheaper overall. Below it, buy the cheap one; above it, do not. *It converts an argument about model quality into a question about your own operations.*
12. A candidate for which another is **at least as accurate and costs no more**, strictly better on one. **Delete it** — there is no trade-off to weigh. `opus-5/low` *and* `opus-5/xhigh` *were dominated in section 15.19.*
13. The **economic** frontier uses point estimates and asks "is anything cheaper and better?". The **defensible** frontier uses evidence and asks "is this established better than everything cheaper?". The second is stricter and shrinks as your evaluation shrinks. `fable-5/medium` *survives the first and fails the second.*

14. **Choose the cheapest candidate whose lower confidence bound clears your quality floor.** The lower bound asks "can I defend this?" rather than "did this come up in my sample?" — which is the question a review or a post-incident analysis will ask. *A point estimate of 0.85 on twenty examples has a lower bound of 0.64.*
15. Because wide intervals mean only the highest scorers have a lower bound above the floor, so the disciplined rule has no cheap option it can defend. *Section 15.21: on 100 examples,* `opus-5/high` *was the only candidate clearing an 0.80 floor. The fix is more examples, not more money.*
16. That **no candidate's quality is established above your floor.** The correct response is a larger evaluation set, not a more expensive model. *In the simulation this happened 6.3% of the time at n=50.*
17. **Claude Opus 5 and Claude Opus 4.8**, research preview, up to **2.5× higher output speed** at **premium pricing** ($10/$50 per MTok), **not available with the Batch API**. *It is the only lever that buys latency without reducing the token count.*
18. Because latency is dominated by `output_tokens × per_token_decode`, and effort moves the token count by up to 5× while the tier moves the per-token rate by much less. *A fast model at high effort can lose to a slow model at low effort.*
19. Any four of: input over 200K (eliminates Haiku 4.5), output over 64K (Haiku 4.5), task budgets required (eliminates Sonnet 5 among current models), `effort` required (Haiku 4.5), interleaved thinking required (Haiku 4.5), non-default `temperature` required (everything from Opus 4.7), knowledge after Feb 2025 without retrieval (Haiku 4.5), data residency, retirement floor. *Check these first; they are free.*
20. Any four of: a new model launches; a model you use is deprecated; your traffic distribution shifts; your failure cost changes; your volume changes by an order of magnitude; a pinned model drifts under stable weights. *Name the triggers in advance so re-opening is routine rather than political.*

15.36 Ten Practical Exercises

1. Add latency to `Candidate` as a p95 measurement and make `decide()` treat it as a constraint rather than an objective.
2. Add a batch flag to every candidate and re-run all seven panels. Which candidates enter the frontier?
3. Implement a false-discovery correction across the ten pairwise comparisons and report how many "established" results survive.
4. Add per-slice accuracy — easy, medium, adversarial — and select on the worst slice rather than the mean.
5. Compute, for each candidate, the evaluation size needed to establish it against the next cheapest. Rank by that number.
6. Add fast mode as a candidate on Opus 5 at $10/$50 and find the latency target at which it enters the frontier.
7. Model failure cost as a distribution rather than a scalar and report the probability that each candidate is cheapest.
8. Extend `total_cost` with a retry policy: on failure, retry once on a dearer candidate. Find the best two-stage cascade.
9. Plot the frontier as accuracy against total cost at three failure costs, and observe the frontier moving.

10. Take a real published benchmark table, apply Chapter 12's five checks and this chapter's interval test, and report how many of its claimed differences are established.

15.37 Five Challenge Labs

1. **The cascade.** Build a two-stage router: a cheap candidate first, escalating to a dearer one when a confidence signal fires. Measure end-to-end accuracy and total cost against every single-model candidate, and find where the cascade beats them both. Note what Chapter 13 says about agreement as a confidence signal before you choose one.
2. **Adaptive selection.** Route per request on a cheap difficulty estimate rather than per workload. Measure the gain over the best fixed choice, and measure how much of it survives a deliberately bad difficulty estimator.
3. **The sequential evaluation.** Instead of a fixed `n`, label examples until the decision is resolved. Implement a sequential test with correct error control, and report the mean examples used against the fixed-`n` design.
4. **Failure cost archaeology.** For a real production system, reconstruct the true cost of a wrong answer from support tickets, review logs and churn data. Compare it against what the team assumed. Report the gap.
5. **Re-run the whole procedure on a new model.** When a model launches, run the harness end to end and produce a one-page decision memo: candidates, evidence, intervals, break-evens, recommendation and re-open trigger. Time yourself. If it takes more than a day, the harness is not finished.

15.38 Ten Interview Questions

1. How would you choose between two Claude models for a new workload?
2. What is wrong with "model A scored 91%, model B scored 90%, so we chose A"?
3. Why is effort part of the selection question rather than a tuning detail afterwards?
4. How large should an evaluation set be?
5. What does a wrong answer cost your system, and how would you find out?
6. When is the cheapest model the most expensive choice?
7. Explain the difference between a dominated candidate and one you cannot justify.
8. Your evaluation says nothing is established. What do you do?
9. When would you buy batch processing instead of a cheaper model?
10. What would make you re-open a model decision you made six months ago?

15.39 Summary

Chapter 14 built the catalogue; this chapter is the procedure that reads it. The question with an answer is not "which model is best" — best is not a property of a model — but "what is the cheapest candidate whose quality I can defend against my bar, on my task, at my volume, within my constraints". The official guidance frames it as capabilities, speed and cost, and adds a fourth criterion that most teams omit:

effort, which trades intelligence for latency and cost within a single model, and which the documentation says is often a better lever than switching models. It offers two starting strategies — efficiency-first from Haiku 4.5, capability-first from Opus 5 — a selection matrix whose every row says "consider starting with", and one unambiguous statement about what actually decides the question: having a good evaluation set is the most important step in the process.

Three reframings do most of the work. A candidate is a (model, effort) pair, because effort crosses the model axis: on the simulated workload, Sonnet 5 at high effort beats Opus 5 at low effort on accuracy while costing $10,800 a month less, and Opus 5 at `xhigh` scores below Opus 5 at `high` while costing $14,000 more. Accuracy is not the metric, because ranking by cost per correct answer inverts the accuracy ranking almost exactly — though that metric carries its own hidden assumption, and it does not simply reward accuracy, since Haiku 4.5 at 50% still beats Opus 5 at 95% on it when the price ratio exceeds the accuracy ratio. And the assumption inside it is the third reframing: dividing by accuracy assumes a wrong answer is free. Write the failure cost down instead, add it to the sum, and the answer moves. At zero the cheapest total-cost candidate is Haiku 4.5 at $6,000 a month; at five pence it is Sonnet 5 at medium effort; at a pound it is Opus 5 at high effort; at five pounds it is Fable 5. Four answers, one set of measurements, one number that was never on the slide. The break-even failure costs against Haiku 4.5 are 4.9p for Sonnet 5 at medium, 18.3p for Opus 5 at high, and 37.7p for Fable 5 at high — thresholds that any process with a human near the failure path clears by an order of magnitude.

The elimination is then mechanical. A candidate is dominated when something is at least as accurate and costs no more, and dominated candidates are deleted rather than debated: two of the ten went that way. The defensible frontier is stricter and more useful, keeping only candidates established better than everything cheaper, and it removed a third — Fable 5 at medium effort, whose 0.4-point lead over Opus 5 at high costs $22,000 a month and is not established even on twenty thousand examples. The decision itself is the floor rule: the cheapest candidate whose lower confidence bound clears your quality bar, using the lower bound because the question a review asks is "can you defend this" rather than "did this come up". The rule is permitted to return nothing, and nothing means measure more rather than spend more.

The simulation exists to score the rule rather than the models, which is something no real evaluation can do, and it produced two results worth carrying. The first is that the naive rule works perfectly and answers the wrong question: as the evaluation grows, picking the top point estimate converges on genuinely identifying the best candidate — the most expensive one — with its bill rising from $45,675 to $67,929 a month for three thousandths of accuracy. The second is the uncomfortable one, and it is about evaluation rather than models. On a hundred examples the intervals are eleven to seventeen points wide, only the top scorers have a lower bound above an 0.80 floor, and so the disciplined rule and the careless one agree — on the expensive answer. A small evaluation set does not merely fail to inform you; it forces you to overbuy. As the set grows, cheaper candidates become defensible and the floor rule's bill falls from $35,172 to $11,993 a month. That $23,179 gap, on identical models with identical true accuracies, is the measured value of a two-thousand-example evaluation set over a fifty-example one. Nothing improved except what could be proved, and what could be proved is what you were allowed to stop paying for.

Some things are not on the frontier at all, and checking them first is free: context and output ceilings, task budget support, effort support, interleaved thinking, sampling-parameter restrictions, knowledge cutoffs, residency and retirement floors all eliminate candidates before a single example is labelled, and Chapter 14's preflight does it in ten seconds. Latency belongs as a constraint rather than an objective,

with the reminder that a faster model at high effort can be slower end-to-end than a slower model at low effort, that fast mode buys up to 2.5× output speed on Opus 5 and Opus 4.8 at premium pricing and not with batch, and that "we cannot afford it" and "we cannot wait" are different sentences — the Batch API's 50% discount moves the whole frontier for anything that tolerates a queue. Finally, a decision is a record rather than an event: the candidate, the evidence, the assumed failure cost, the date and the trigger to re-open, stored where Chapter 14's registry keeps it so that when a new model launches, a model is deprecated, traffic shifts, the failure cost changes, volume moves an order of magnitude, or a pinned model drifts, the answer is one evaluation run and one line rather than another eleven-minute meeting.

KEY TAKEAWAYS

1. **The question is "cheapest defensible candidate", not "best model".** Best is not a property of a model.
2. **A candidate is a (model, effort) pair.** Fixed-effort comparisons search a line through a plane.
3. **The evaluation set is the most important step** — your task, your distribution, your metric, chosen first.
4. **Size for the difference that would change your mind.** Halving the gap quadruples the sample.
5. **Cost per correct answer inverts the accuracy ranking** — and assumes a wrong answer is free.
6. **Write down the failure cost.** Break-evens against Haiku 4.5 were 4.9p, 18.3p and 37.7p.
7. **Delete dominated candidates.** Two of ten. Cheaper and better is not a trade-off.
8. **The defensible frontier removes what you cannot prove** — $22,000 a month for 0.4 of a point, unestablished at n=20,000.
9. **The floor rule uses the lower bound**, and "not established" means measure more, not spend more.
10. **A 2,000-example evaluation was worth $23,179 a month** over a 50-example one. Small evaluations force you to overbuy.

15.40 Glossary

Term	Definition
Break-even failure cost	`F* = (cost_d − cost_c) / (acc_d − acc_c)`; the failure price at which the dearer candidate becomes cheaper overall.
Candidate	A shippable configuration: a model plus an effort level, and any other settings you would deploy.
Capability-first	The documented strategy of starting with Claude Opus 5 and optimising downward.
Cost per correct answer	Request cost divided by accuracy. Valid only when a wrong answer is free to discard.
Defensible frontier	Candidates established better than everything cheaper than them.
Dominated candidate	One for which another is at least as accurate and costs no more. Delete rather than debate.
Economic frontier	Candidates that nothing dominates, computed from point estimates.

Term	Definition
Efficiency-first	The documented strategy of starting with Claude Haiku 4.5 and upgrading only for capability gaps.
Established difference	A gap that survives a two-proportion test at your confidence level.
Failure cost	What one wrong answer costs downstream: retry, review, contact, refund, churn.
Fast mode	Research preview on Opus 5 and Opus 4.8; up to 2.5× output speed at premium pricing, not with batch.
Floor rule	Choose the cheapest candidate whose lower confidence bound clears the quality floor.
Quality floor	The accuracy bar the workload requires, set before measurement.
Total cost	`inference + (1 − accuracy) × failure cost`. The honest sum.

15.41 Further Reading

Primary sources. `platform.claude.com/docs/en/about-claude/models/choosing-a-model` is the source for the four criteria, both starting strategies, the selection matrix and the statement that a good evaluation set is the most important step; `.../build-with-claude/effort` for the level-by-level guidance and the warning that settings do not transfer between models; `.../build-with-claude/fast-mode` for the research preview; `.../about-claude/pricing` for the complete rate card including batch and caching; and `.../test-and-evaluate/develop-tests` for Anthropic's own guidance on building the evaluation set this chapter depends on. The per-model release notes — *What's new in Claude Opus 5*, *What's new in Claude Sonnet 5* — are worth reading before a shortlist rather than after.

Where to go next in this book. Chapter 16 turns the chosen candidate into a request, and Chapter 17 covers the input modalities every candidate supports. Chapter 18 is thinking as an API surface, which is where the effort axis of this chapter becomes code. Chapter 19 covers rate limits and quotas — the constraint that can eliminate a candidate after you have chosen it. **Chapter 24 is the direct sequel**: it turns the evaluation set built here into a regression suite that runs continuously, which is what makes the re-open triggers of section 15.14 cheap enough to honour. Chapter 30 covers batch processing and prompt caching, the two levers that move the frontier furthest. Chapter 50 covers orchestrator–subagent topologies, where selection becomes per-role rather than per-system, and Chapter 51 covers reflection and self-healing, which is section 15.37's cascade built properly. Chapter 57 is the observability that keeps the failure cost of section 15.23 an estimate you update rather than a guess you made once.